Student details:

Student ID : 1115201800122

FULL NAME : BANTAK EMIL

Part A of the project:

A1-A2:

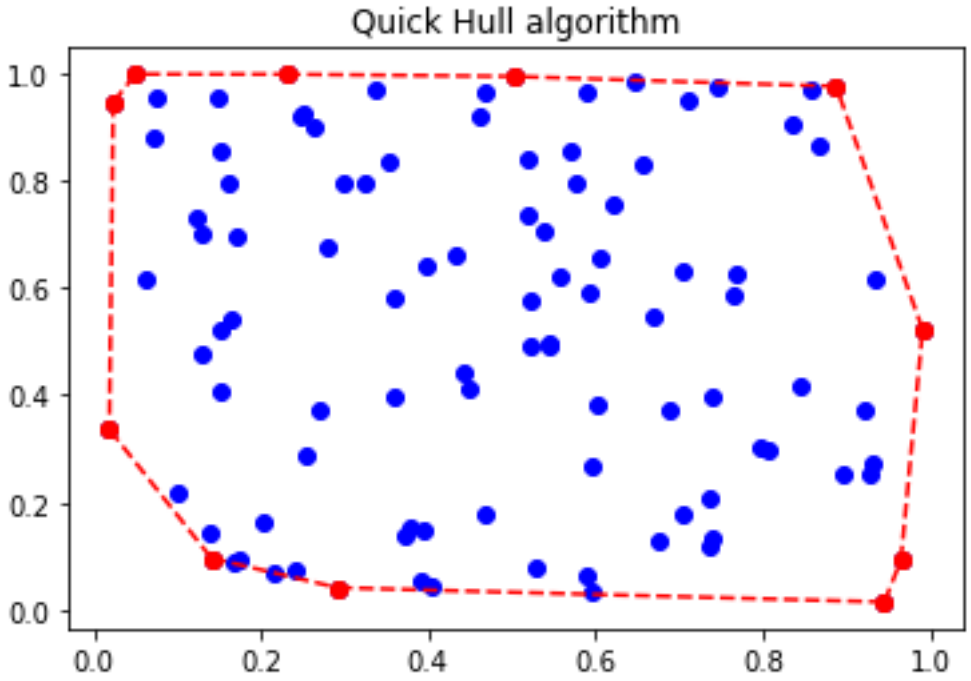
=> For the following set of 100 points, where each point was generated randomly using the function A_part.generate_n_random_points(100, 2)

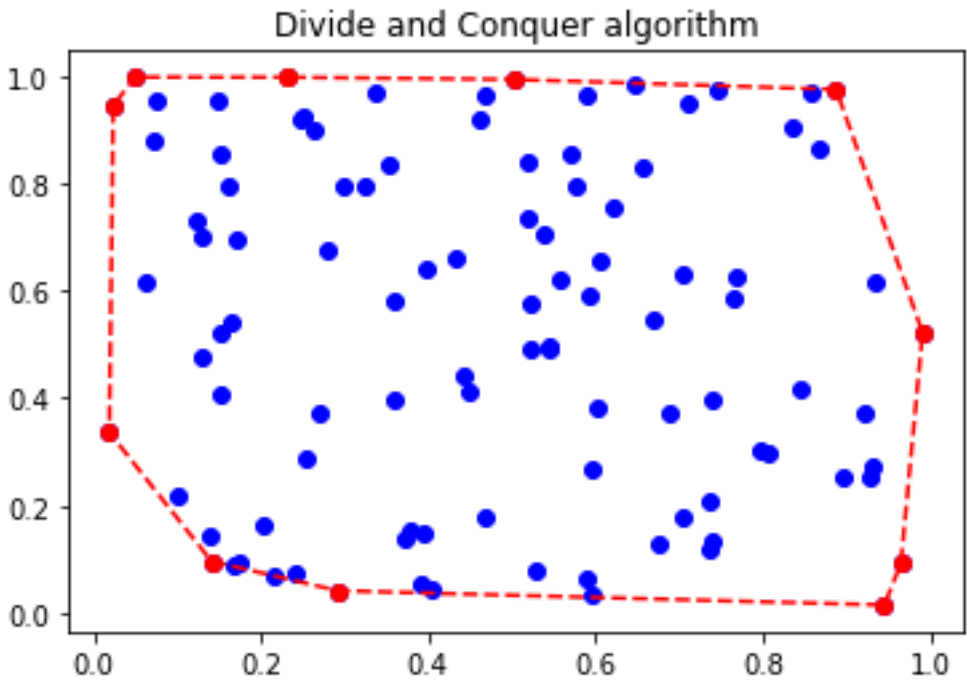
```
points = np.array([
    [0.93147929, 0.27226631],
    [0.93448094, 0.61595293],
    [0.60443924, 0.6581045],
    [0.25403866, 0.28546695],
    [0.59445035, 0.26929064],
    [0.12884231, 0.70066496],
    [0.23183824, 0.99790449],
    [0.15162308, 0.40962568],
    [0.7635474, 0.5886206],
    [0.73609261, 0.20791308],
    [0.25160527, 0.92327522],
    [0.92776741, 0.254034],
    [0.3962927, 0.64292784],
    [0.54318177, 0.49305957],
    [0.46193849, 0.91953441],
    [0.74590625, 0.97520922],
    [0.40296685, 0.04508187],
    [0.46694494, 0.96299019],
    [0.44181965, 0.44200838],
    [0.32436679, 0.79679759],
    [0.51739351, 0.84145935],
    [0.20384024, 0.16475931],
    [0.65655798, 0.83020063],
    [0.16462795, 0.54207393],
    [0.12971134, 0.47592225],
    [0.39297108, 0.14842349],
    [0.62153246, 0.75323935],
    [0.33802243, 0.96746339],
    [0.71183667, 0.9488336],
    [0.09877945, 0.21690739],
    [0.59006033, 0.06414008],
    [0.73757022, 0.11885667],
    [0.89570807, 0.25415001],
    [0.5697079, 0.85661201],
    [0.83663927, 0.90638199],
    [0.24752908, 0.92011486],
    [0.44959007, 0.41250029],
    [0.26904099, 0.37358171],
```

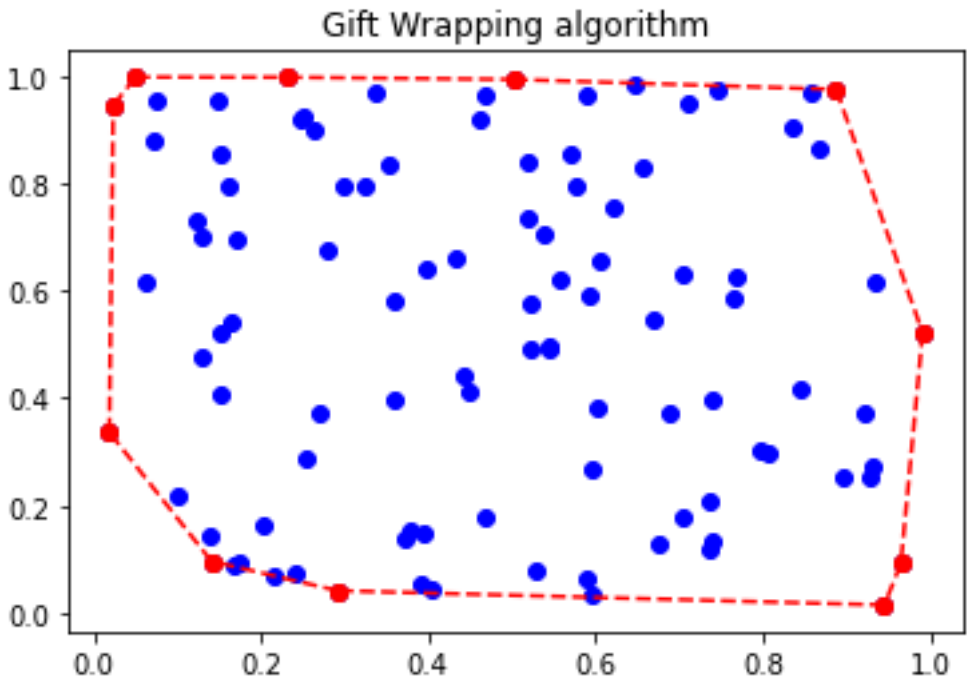
[0.13920285, 0.14438533],
[0.29160976, 0.04131683],
[0.17111403, 0.69529807],
[0.15049755, 0.85530383],
[0.59403886, 0.03250771],
[0.67485005, 0.13100408],
[0.73799407, 0.39882031],
[0.68883923, 0.37380986],
[0.26384719, 0.8985391],
[0.84530555, 0.41791875],
[0.43229435, 0.65900811],
[0.24184507, 0.07153227],
[0.07467331, 0.95371286],
[0.16190968, 0.7949359],
[0.46898516, 0.17828398],
[0.35252582, 0.8349938],
[0.79577634, 0.30375255],
[0.88748217, 0.97537955],
[0.07172741, 0.87840939],
[0.99090898, 0.52222277],
[0.96493973, 0.09219253],
[0.52911218, 0.08137454],
[0.35862558, 0.39569947],
[0.73978602, 0.13480078],
[0.80500675, 0.29669075],
[0.57753759, 0.7973416],
[0.60179035, 0.38124844],
[0.16684825, 0.08690103],
[0.70482977, 0.18054004],
[0.12318069, 0.73235335],
[0.35933837, 0.58345324],
[0.04983997, 0.99824477],
[0.55665907, 0.6191218],
[0.94515905, 0.01468197],
[0.39123243, 0.05499007],
[0.0231741, 0.94552682],
[0.17389323, 0.09539619],
[0.29780635, 0.79304771],
[0.58976523, 0.96636277],
[0.06102021, 0.61623619],
[0.92225562, 0.37227637],
[0.52118565, 0.57424999],
[0.76855796, 0.62391902],
[0.15067848, 0.52078799],
[0.66961293, 0.54559214],
[0.5382037, 0.70566423],
[0.21651136, 0.06996702],
[0.37808611, 0.15286844],
[0.85851731, 0.97054308],
[0.64677749, 0.98478398],
[0.86679094, 0.86441424],
[0.70478294, 0.62882976],

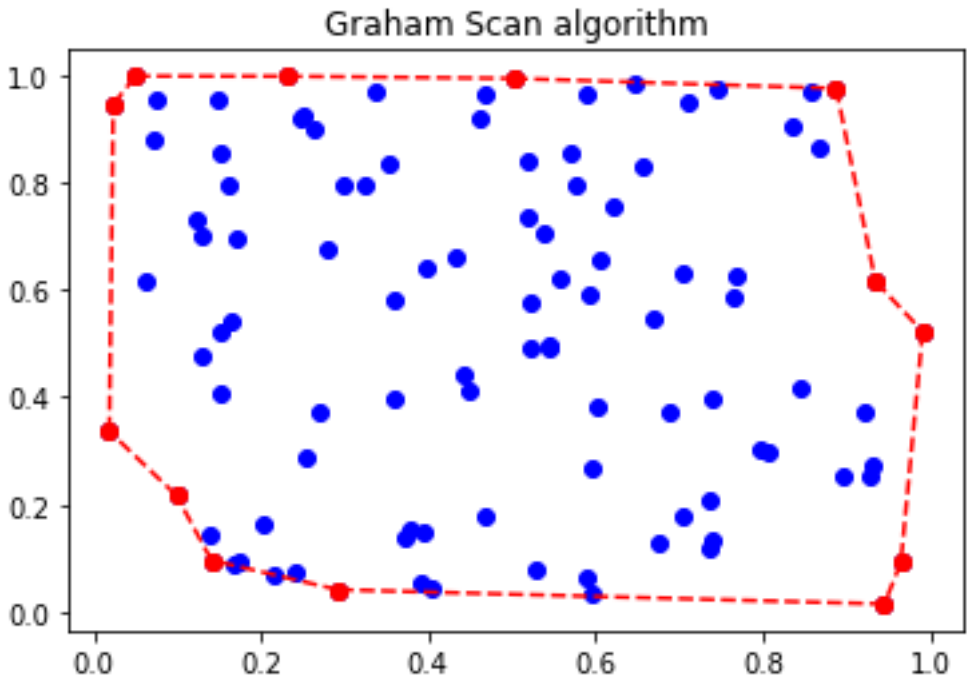
[0.54381601, 0.49476383],
[0.59328719, 0.58933277],
[0.50132689, 0.99396054],
[0.14060691, 0.09625285],
[0.01856888, 0.33628721],
[0.14852242, 0.95447917],
[0.27935191, 0.6780794],
[0.37176963, 0.14002385],
[0.52282242, 0.49065627],
[0.51811794, 0.73548264]])

we get the following figures for each algorithm :




Divide and Conquer algorithm
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


Gift Wrapping algorithm
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0



We observe from the figures that the convex hulls produced by each algorithm except Graham Scan are the same and the points that belong to the convex hull are the following :

[[0.01856888 0.33628721]
 [0.14060691 0.09625285]
 [0.29160976 0.04131683]
 [0.94515905 0.01468197]
 [0.96493973 0.09219253]
 [0.99090898 0.52222277]
 [0.88748217 0.97537955]
 [0.50132689 0.99396054]
 [0.23183824 0.99790449]
 [0.04983997 0.99824477]
 [0.0231741 0.94552682]]

Whereas the Graham Scan algorithm returns a few more points belonging to the convex hull, the points that belong to the convex hull with the Graham Scan algorithm are the following :

[[0.01856888 0.33628721]
 [0.0231741 0.94552682]
 [0.04983997 0.99824477]
 [0.23183824 0.99790449]
 [0.50132689 0.99396054]
 [0.88748217 0.97537955]
 [0.93448094 0.61595293]
 [0.99090898 0.52222277]
 [0.96493973 0.09219253]
 [0.94515905 0.01468197]
 [0.29160976 0.04131683]
 [0.14060691 0.09625285]

[0.09877945 0.21690739]]

These create some “dents” but this is consistent with the theory of the course for the Graham Scan algorithm

Note 1): the implementation of the algorithms also takes into account the case where at a given moment the last three points belonging to the convex hull are collinear, then the middle point is not taken into account. This is why there are various points in the figures that lie on the convex hull but are not counted (i.e. they remain a ‘blue dot’ and not a ‘red dot’)

Note 2): the implementation of the Divide_and_Conquer algorithm is largely based on the following link :
https://www.geeksforgeeks.org/convex-hull-using-divide-and-conquer-algorithm/

=> ***Regarding the computation times of the convex hull:***

Graham_Scan computes it in 0.06038403511047363 secs

Gift_Wrapping computes it in 0.05990195274353027 secs

Divide_and_Conquer computes it in 0.056380510330200195 secs

Quick_Hull computes it in 0.061322927474975586 secs

The fastest is Divide_and_Conquer while the slowest is Quick_Hull which however is consistent with the time complexities according to the theory of the course. We also observe that Graham_Scan is not that fast and a contributing factor is that there are some “dents” in the convex hull for this set of points. In the case of a different set of points that would not create any “dents” in the convex hull Graham_Scan has about the same time as Divide_and_Conquer which agrees with the theory of the course.

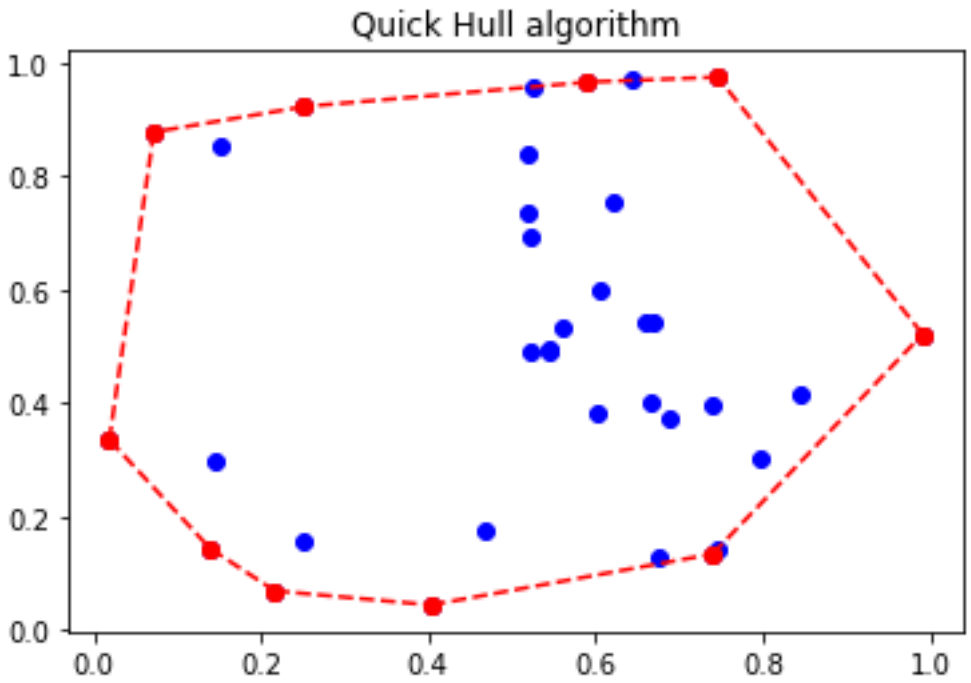
A3:

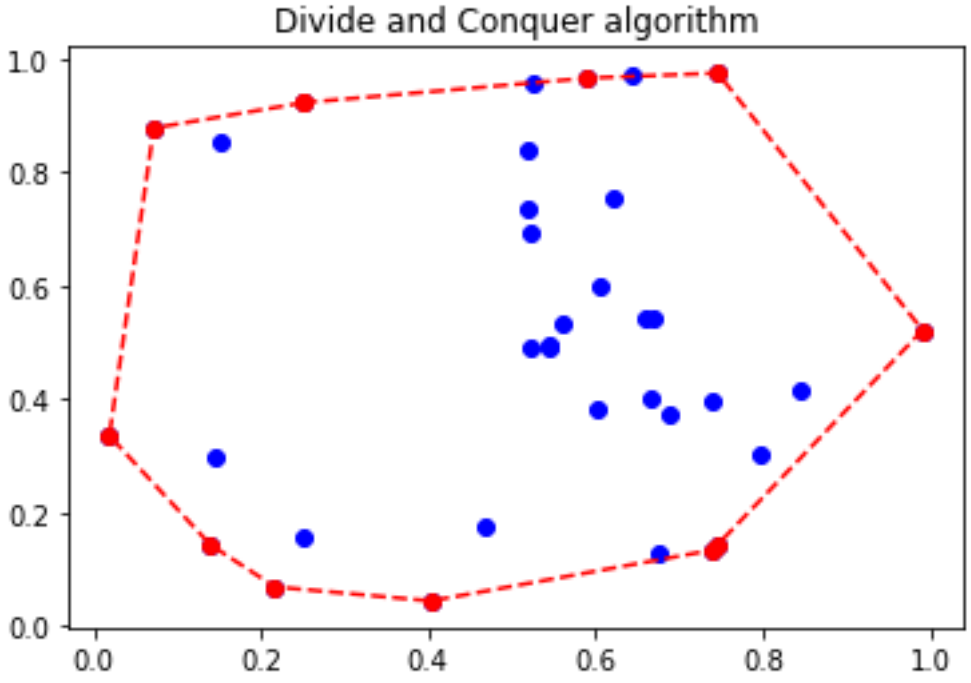
For the following new set of points which was created by randomly selecting 25 points from the initial set of points and in which there are at least 10 points that are collinear with some of the 25 points

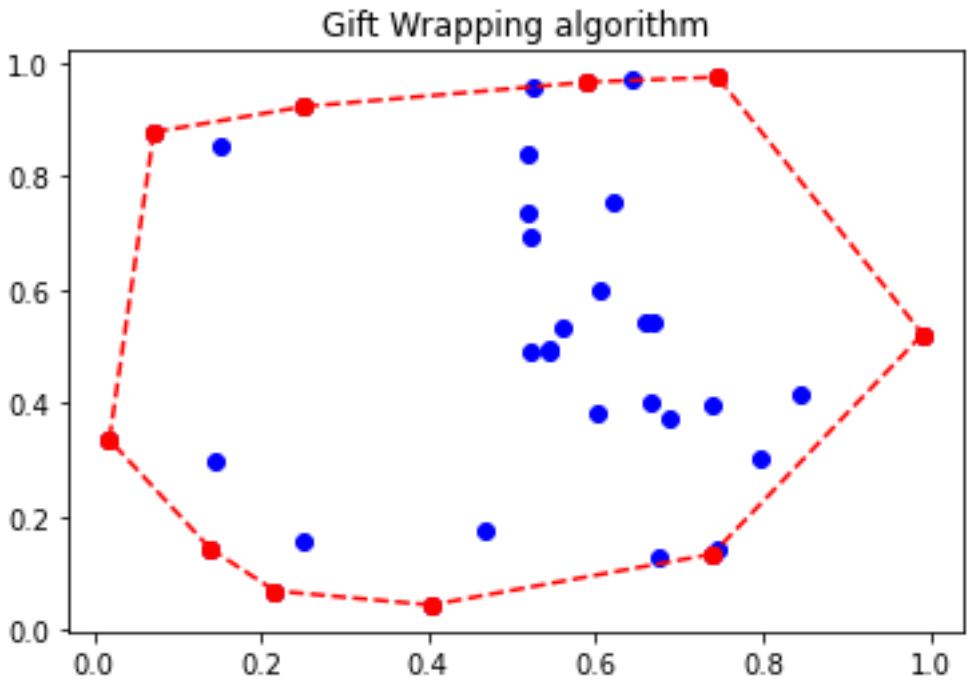
```
new_points = np.array([
   [0.21651136, 0.06996702],
   [0.99090898, 0.52222277],
   [0.52282242, 0.49065627],
   [0.67485005, 0.13100408],
   [0.25160527, 0.92327522],
   [0.73799407, 0.39882031],
   [0.66961293, 0.54559214],
   [0.54318177, 0.49305957],
   [0.01856888, 0.33628721],
   [0.07172741, 0.87840939],
   [0.73978602, 0.13480078],
   [0.46898516, 0.17828398],
```

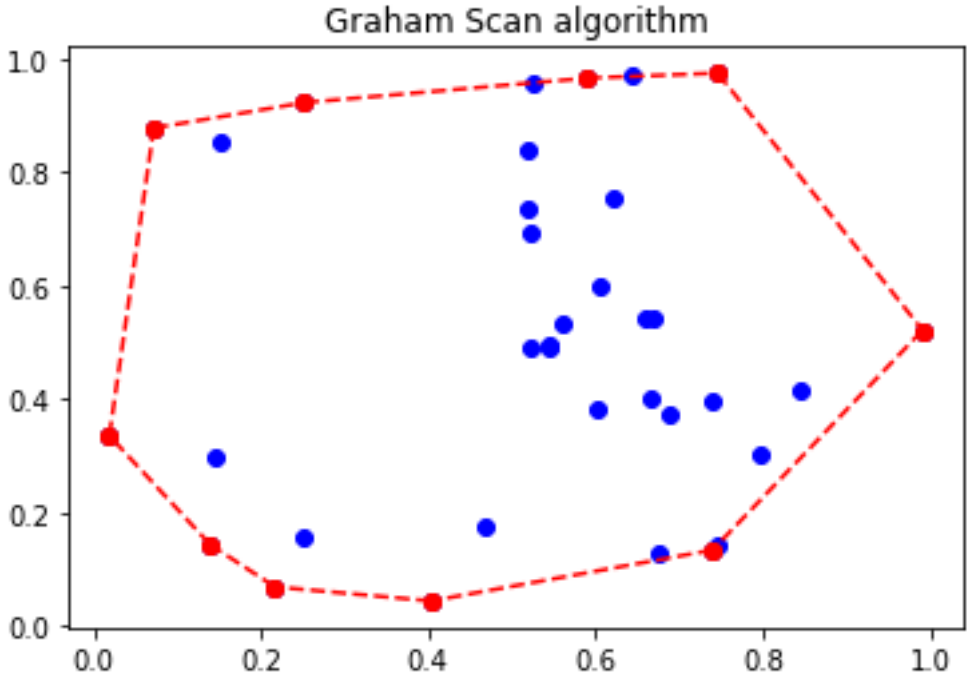
[0.74590625, 0.97520922],
[0.62153246, 0.75323935],
[0.15049755, 0.85530383],
[0.13920285, 0.14438533],
[0.51811794, 0.73548264],
[0.79577634, 0.30375255],
[0.58976523, 0.96636277],
[0.60179035, 0.38124844],
[0.84530555, 0.41791875],
[0.54381601, 0.49476383],
[0.51739351, 0.84145935],
[0.68883923, 0.37380986],
[0.40296685, 0.04508187],
[0.14449103, 0.30110834],
[0.74504336, 0.14291158],
[0.6649915, 0.40377895],
[0.52259625, 0.69216743],
[0.56065267, 0.53461148],
[0.6594587, 0.54137302],
[0.60559473, 0.60087443],
[0.64370058, 0.96941857],
[0.24935394, 0.15570787],
[0.52436075, 0.95802908]])

we get the following figures for each algorithm :




Divide and Conquer algorithm
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


Gift Wrapping algorithm
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0



We observe from the figures that the convex hulls produced by each algorithm are the same and the points that belong to the convex hull are the following :

[[0.01856888 0.33628721]
[0.13920285 0.14438533]
[0.21651136 0.06996702]
[0.40296685 0.04508187]
[0.73978602 0.13480078]
[0.99090898 0.52222277]
[0.74590625 0.97520922]
[0.58976523 0.96636277]
[0.25160527 0.92327522]
[0.07172741 0.87840939]]

For this set of points we do not observe any “dent” being created by the Graham Scan algorithm and that is why we get the same set of convex hull points as with the other three algorithms. This is consistent with the theory of the course for the Graham Scan algorithm i.e. sometimes some “dents” may be created.

These four algorithms are not particularly affected by the existence of some collinear points, in particular for the convex hull the implementation of the algorithms also takes into account the case where at a given moment the last three points belonging to the convex hull are collinear, then the middle point is not taken into account. This is why there are various points in the figures that lie on the convex hull but are not counted (i.e. they remain a ‘blue dot’ and not a ‘red dot’)

=> ***Regarding the computation times of the convex hull:***

Graham_Scan computes it in 0.05727505683898926 secs

Gift_Wrapping computes it in 0.05789303779602051 secs

Divide_and_Conquer computes it in 0.05498003959655762 secs

Quick_Hull computes it in 0.060596466064453125 secs

A4:

A gif has been created (it is located in the same folder as the project). Using the same set of points as in questions ***A1-A2*** and the Gift_Wrapping algorithm.

A5-A6:

=> **First to compute the convex hull in 3D the ready-made scipy function ConvexHull(...) has been used which internally implements the Quick Hull algorithm**

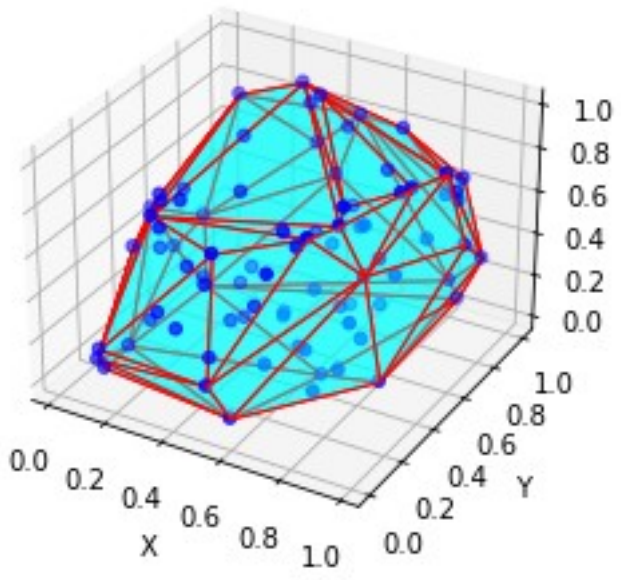
With that said for the following set of points

```
points_3d = np.array([
   [0.07471274, 0.30985218, 0.45929452],
   [0.13839123, 0.01118205, 0.20936368],
   [0.86794521, 0.39900953, 0.65193642],
   [0.28860546, 0.10369138, 0.37545234],
   [0.68970513, 0.87764746, 0.87646618],
   [0.13483434, 0.38217775, 0.64491161],
   [0.65880102, 0.10542189, 0.69952279],
   [0.23255595, 0.75009529, 0.1252761],
   [0.09151819, 0.70027621, 0.01320628],
   [0.50184368, 0.0631782, 0.75749565],
   [0.85750091, 0.17114192, 0.94692455],
   [0.34427932, 0.96420817, 0.631222],
   [0.01648191, 0.50743363, 0.4932137],
   [0.30088257, 0.25322026, 0.49213754],
   [0.30731633, 0.33937146, 0.3514075],
   [0.45325609, 0.56824009, 0.12243702],
   [0.92469822, 0.38945053, 0.60711707],
   [0.38444578, 0.85375841, 0.89831643],
   [0.87154153, 0.89443507, 0.59773578],
   [0.28312631, 0.13119721, 0.74358865],
   [0.93140492, 0.76131204, 0.74584466],
   [0.69935905, 0.26110553, 0.26007918],
   [0.16034397, 0.57505624, 0.17087244],
   [0.26148652, 0.12103368, 0.79734645],
   [0.96230062, 0.41130013, 0.99639616],
   [0.11172202, 0.54646151, 0.58734526],
   [0.82354649, 0.23540642, 0.23945051],
```

[0.30683062, 0.57557625, 0.03169089],
[0.01354956, 0.6407031, 0.21710704],
[0.73467413, 0.66349846, 0.20393589],
[0.58611744, 0.01553185, 0.07324357],
[0.67745876, 0.24656061, 0.73370055],
[0.48461534, 0.9457789, 0.3918814],
[0.82330774, 0.25907776, 0.95886478],
[0.48112634, 0.99327736, 0.79699715],
[0.58067646, 0.63901447, 0.44725795],
[0.8887136, 0.65990352, 0.57762094],
[0.47831798, 0.72121176, 0.40797353],
[0.45022398, 0.79469622, 0.99301385],
[0.52888462, 0.43186436, 0.61885545],
[0.65850052, 0.09234183, 0.29783287],
[0.66821642, 0.82364429, 0.7081786],
[0.90779437, 0.33202638, 0.15069477],
[0.60139953, 0.71963603, 0.11325736],
[0.55319946, 0.38825834, 0.26981174],
[0.23707635, 0.7006678, 0.7861667],
[0.64547229, 0.31791906, 0.21304495],
[0.93017166, 0.80774274, 0.77677564],
[0.95670573, 0.77439632, 0.50072126],
[0.47238485, 0.93083522, 0.77128708],
[0.64760127, 0.68898969, 0.51978151],
[0.9058108, 0.45277813, 0.92802508],
[0.39712864, 0.99435539, 0.48242201],
[0.92540749, 0.96257295, 0.30182162],
[0.06114187, 0.74401293, 0.3187013],
[0.50888392, 0.2926235, 0.36291488],
[0.12976489, 0.0122541, 0.15873403],
[0.08404658, 0.45014804, 0.36219121],
[0.37002714, 0.70187948, 0.39405108],
[0.80008984, 0.99224466, 0.12448438],
[0.13895652, 0.29905873, 0.1439939],
[0.97042105, 0.11232393, 0.81113907],
[0.79238085, 0.10381125, 0.91200069],
[0.31595292, 0.4783931, 0.08607164],
[0.51076671, 0.00684848, 0.19754117],
[0.91524075, 0.72323266, 0.74910273],
[0.17429153, 0.77911798, 0.91253695],
[0.34775832, 0.10824249, 0.32047866],
[0.44817556, 0.13064018, 0.21200705],
[0.23754127, 0.66996947, 0.10547693],
[0.98840882, 0.57874387, 0.96269153],
[0.12499961, 0.05744753, 0.08160438],
[0.07028638, 0.27675627, 0.00817341],
[0.35376307, 0.47953887, 0.70953913],
[0.79275195, 0.43305236, 0.22811195],
[0.35891106, 0.84796284, 0.98983513],
[0.8644391, 0.31136595, 0.85926709],
[0.27339051, 0.26687235, 0.77599137],
[0.70217752, 0.47123833, 0.31651291],

[0.20478751, 0.26055698, 0.78266587],
[0.63860625, 0.42375927, 0.40261156],
[0.85820624, 0.93737255, 0.09707516],
[0.7609706, 0.1984202, 0.13945718],
[0.66781588, 0.4151042, 0.01979494],
[0.83793791, 0.27552816, 0.43114463]])

we get the following figure :



and the set of points that belong to the convex hull is the following :

[[0.07471274 0.30985218 0.45929452]
[0.13839123 0.01118205 0.20936368]
[0.68970513 0.87764746 0.87646618]
[0.09151819 0.70027621 0.01320628]
[0.50184368 0.0631782 0.75749565]
[0.85750091 0.17114192 0.94692455]
[0.34427932 0.96420817 0.631222]
[0.01648191 0.50743363 0.4932137]
[0.87154153 0.89443507 0.59773578]
[0.26148652 0.12103368 0.79734645]
[0.96230062 0.41130013 0.99639616]
[0.01354956 0.6407031 0.21710704]
[0.58611744 0.01553185 0.07324357]
[0.82330774 0.25907776 0.95886478]
[0.48112634 0.99327736 0.79699715]
[0.45022398 0.79469622 0.99301385]
[0.90779437 0.33202638 0.15069477]
[0.93017166 0.80774274 0.77677564]
[0.95670573 0.77439632 0.50072126]
[0.39712864 0.99435539 0.48242201]

[0.92540749 0.96257295 0.30182162]
[0.06114187 0.74401293 0.3187013]
[0.12976489 0.0122541 0.15873403]
[0.80008984 0.99224466 0.12448438]
[0.97042105 0.11232393 0.81113907]
[0.79238085 0.10381125 0.91200069]
[0.51076671 0.00684848 0.19754117]
[0.17429153 0.77911798 0.91253695]
[0.98840882 0.57874387 0.96269153]
[0.12499961 0.05744753 0.08160438]
[0.07028638 0.27675627 0.00817341]
[0.35891106 0.84796284 0.98983513]
[0.20478751 0.26055698 0.78266587]
[0.85820624 0.93737255 0.09707516]
[0.66781588 0.4151042 0.01979494]]

Part B of the project:

=> **In this part of the exercise the ready-made scipy function linprog(...) has been used**

And as the solution for this part of the project we get the following output in a terminal window:

```
--------------------------------------------------------------------------------
OUTPUT of the B part of the project
X1 solution: 12.0
X2 solution: 4.0
MaxValue : 12.0
--------------------------------------------------------------------------------
```

Part C of the project:

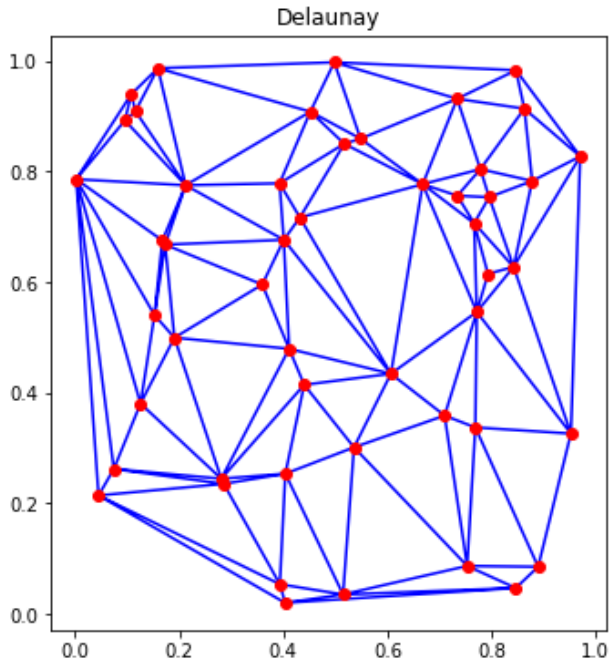
=> **First for the Delaunay triangulation and the Voronoi diagram the ready-made scipy functions Delaunay(...) and Voronoi(...) have been used respectively.**

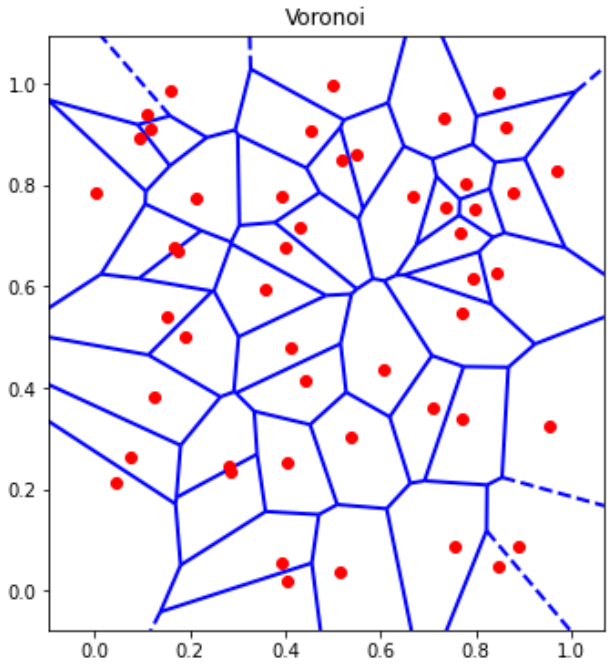
> For the following set of 50 random points

```
points = np.array([
   [0.35948799, 0.59556889],
   [0.12578288, 0.38044272],
   [0.84667675, 0.98283836],
   [0.17314647, 0.66770123],
   [0.28087975, 0.24472862],
   [0.51470788, 0.03549435],
   [0.00232315, 0.78589411],
   [0.77838305, 0.804075],
   [0.28560743, 0.23537891],
   [0.77148607, 0.54601779],
   [0.10818133, 0.93965129],
   [0.45190313, 0.90793187],
   [0.84217325, 0.62752592],
```

[0.54833339, 0.8591002],
[0.43181524, 0.71680152],
[0.70971278, 0.35878548],
[0.87789602, 0.78276103],
[0.19093742, 0.4995833],
[0.73412888, 0.75608802],
[0.44007009, 0.41441951],
[0.39334771, 0.05405968],
[0.40356664, 0.0203562],
[0.16773747, 0.67518401],
[0.09576465, 0.89204908],
[0.15265528, 0.54027077],
[0.39283873, 0.77848136],
[0.84629261, 0.04719171],
[0.79302427, 0.61482139],
[0.97008697, 0.82714017],
[0.95224398, 0.32625744],
[0.40091548, 0.67633476],
[0.5171416, 0.84996781],
[0.88931649, 0.08554083],
[0.15918579, 0.98648206],
[0.86284485, 0.91337803],
[0.75299635, 0.08705822],
[0.07457774, 0.2622895],
[0.49780617, 0.99717151],
[0.5357253, 0.30141067],
[0.04522391, 0.21446036],
[0.40389571, 0.25374604],
[0.41118946, 0.4794816],
[0.11602082, 0.90885576],
[0.79518464, 0.75337885],
[0.76869586, 0.3373562],
[0.21243948, 0.77500814],
[0.73372546, 0.93124209],
[0.60663221, 0.43436441],
[0.76644312, 0.70632648],
[0.6675973, 0.77687295]])

we get as a figure





With a total time of 0.11316823959350586 secs

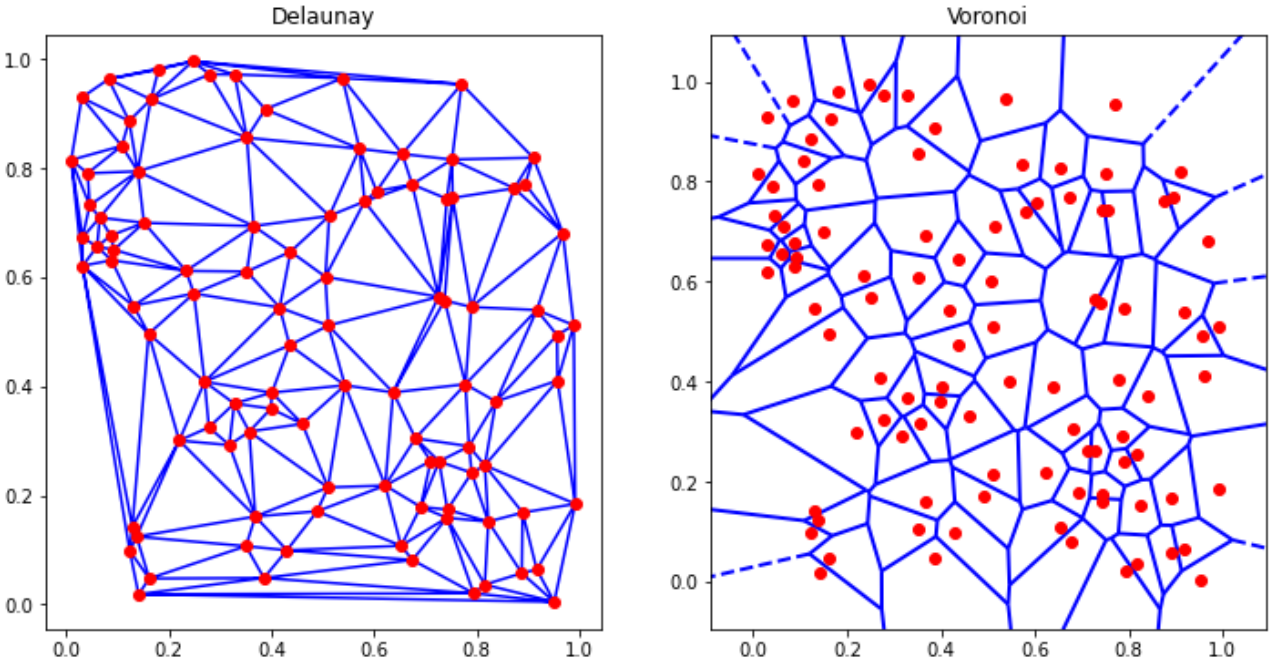
> For the following set of 100 random points

```
points = np.array([
    [0.69109182, 0.17877036],
    [0.13911237, 0.79396112],
    [0.81630078, 0.03623742],
    [0.26945086, 0.40816396],
    [0.78541911, 0.28988285],
    [0.16133691, 0.04742597],
    [0.91784206, 0.53978611],
    [0.35190918, 0.61016964],
    [0.08615011, 0.67629022],
    [0.88852291, 0.05723427],
    [0.24628123, 0.995701],
    [0.17964001, 0.98085848],
    [0.43536298, 0.47518314],
    [0.10775757, 0.84060672],
    [0.78997226, 0.54512851],
    [0.67458766, 0.77000269],
    [0.9517236, 0.0046292],
    [0.05964987, 0.65582191],
    [0.21983113, 0.30041805],
    [0.50510548, 0.60066858],
    [0.8163869, 0.25491643],
    [0.00903911, 0.81481424],
    [0.62197466, 0.21793158],
    [0.6051057, 0.75768031],
    [0.15106539, 0.69954754],
    [0.83800963, 0.37181119],
```

[0.27917766, 0.97197434],
[0.14054893, 0.01859299],
[0.12835223, 0.1423216],
[0.95823895, 0.41042449],
[0.75224626, 0.81557081],
[0.74327914, 0.17384281],
[0.35015409, 0.10678573],
[0.27786881, 0.32465192],
[0.08769594, 0.63099373],
[0.91776517, 0.06395631],
[0.3988678, 0.35893667],
[0.32728057, 0.36821395],
[0.71077934, 0.26293383],
[0.67513761, 0.08141665],
[0.24855468, 0.57000172],
[0.72703739, 0.26214533],
[0.4158861, 0.54431685],
[0.36838753, 0.1611541],
[0.38515239, 0.0483244],
[0.40070608, 0.38828634],
[0.79449587, 0.02032145],
[0.13666513, 0.12420769],
[0.77105767, 0.95381265],
[0.77862423, 0.40367389],
[0.89527522, 0.77028942],
[0.82312275, 0.1516963],
[0.73776759, 0.55733067],
[0.48905027, 0.1706957],
[0.08183015, 0.96302218],
[0.35655951, 0.31684575],
[0.43004436, 0.09920825],
[0.04517404, 0.73170803],
[0.51275653, 0.71220995],
[0.51072634, 0.51125595],
[0.09243298, 0.65025062],
[0.74248566, 0.15879122],
[0.3650144, 0.693629],
[0.68088431, 0.30445485],
[0.65289254, 0.10740195],
[0.31761007, 0.29210681],
[0.65505541, 0.82585892],
[0.53844346, 0.96466425],
[0.54332344, 0.40173597],
[0.23314454, 0.61145088],
[0.16442311, 0.92575939],
[0.03104959, 0.67276213],
[0.04105525, 0.79034604],
[0.74230548, 0.74358751],
[0.1601915, 0.49606173],
[0.43732412, 0.64676177],
[0.32789589, 0.97154522],
[0.91066936, 0.81900388],

[0.51021212, 0.21448815],
[0.9573008, 0.49366468],
[0.89008062, 0.16829515],
[0.75361281, 0.74524986],
[0.46013487, 0.33105477],
[0.13066729, 0.54755574],
[0.6373542, 0.38859325],
[0.99276739, 0.18422903],
[0.02957578, 0.92910635],
[0.72597319, 0.56384396],
[0.5816728, 0.73876653],
[0.06475763, 0.70920216],
[0.57176948, 0.83530122],
[0.873859, 0.76290662],
[0.79077135, 0.24114196],
[0.12336337, 0.09783758],
[0.03102325, 0.62065597],
[0.12203113, 0.88628826],
[0.98993575, 0.51186164],
[0.96856744, 0.68119483],
[0.35194217, 0.85561534],
[0.38765925, 0.90783066]])

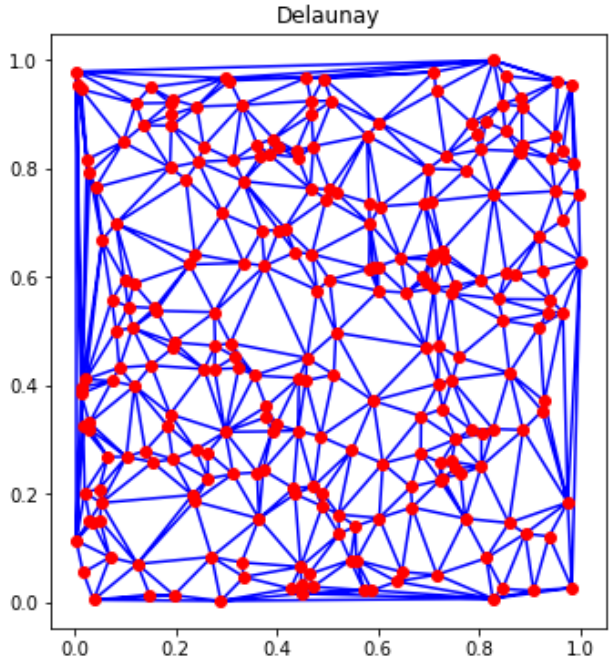
we get as a figure

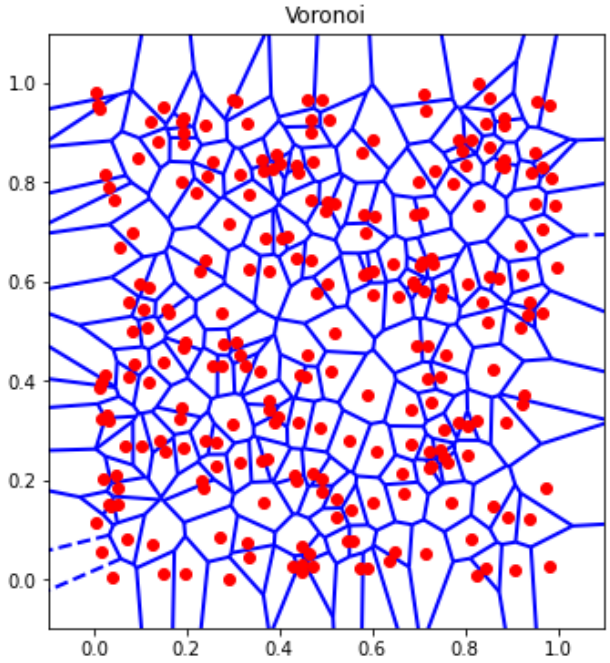


With a total time of 0.11672782897949219 secs

> For a set of 250 random points

we get as a figure

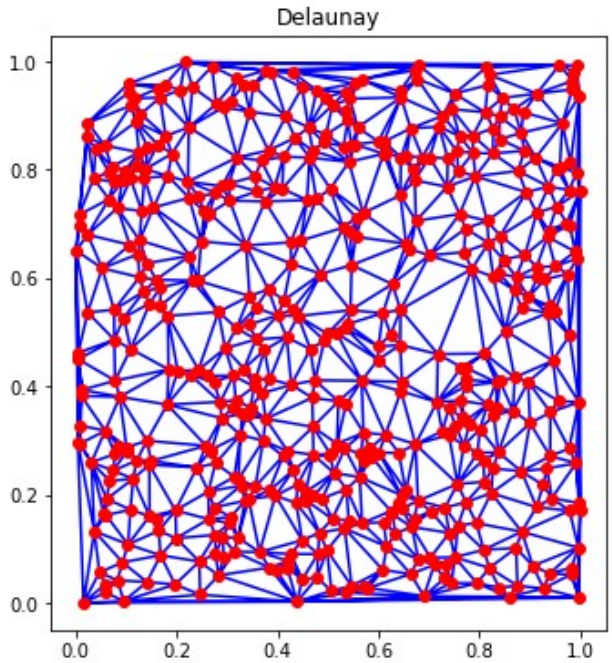


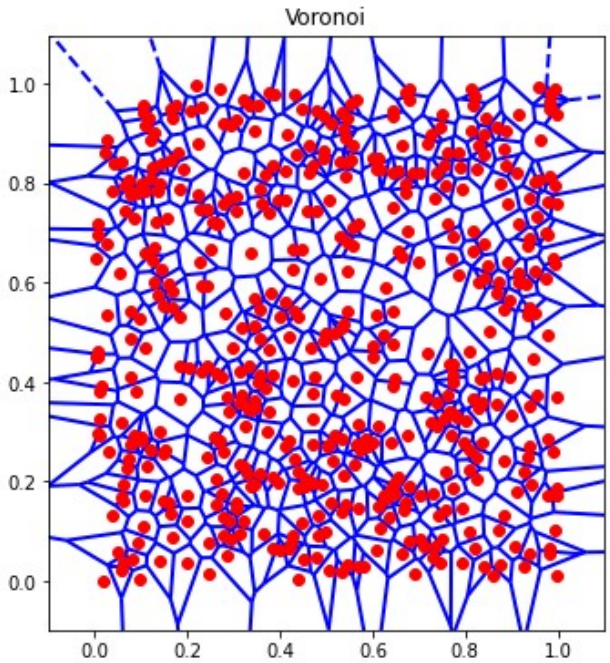


With a total time of 0.1273338794708252 secs

> For a set of 500 random points

we get as a figure





With a total time of 0.18764281272888184 secs

=> We observe that as the number of points increases the total execution time also increases which according to the theory of the course is to be expected

Part D of the project:

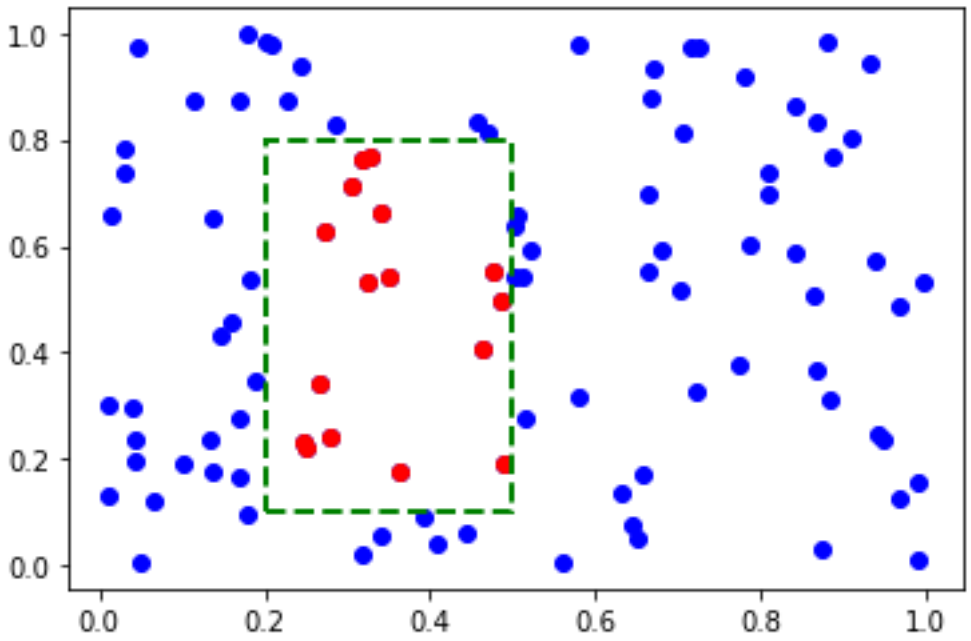
> For the following set of 100 random points

```
points = np.array([
    [0.03826764, 0.29727545],
    [0.40975177, 0.04318333],
    [0.86698024, 0.83565627],
    [0.66536958, 0.69768924],
    [0.47077749, 0.81576946],
    [0.8795336,  0.98316795],
    [0.67173427, 0.93397238],
    [0.67873236, 0.59169618],
    [0.65784003, 0.17307355],
    [0.1695267,  0.87460542],
    [0.55948969, 0.00825489],
    [0.13306935, 0.23707014],
    [0.47544821, 0.55444864],
    [0.06456101, 0.12185814],
    [0.31688486, 0.76546225],
    [0.50105476, 0.63960731],
    [0.94791523, 0.2385098 ],
    [0.88458014, 0.31183746],
    [0.7164212,  0.97367375],
    [0.50666717, 0.65664723],
    [0.1786864,  0.99747843],
    [0.2475788,  0.23090367],
    [0.01172699, 0.30123491],
    [0.99210039, 0.01153587],
    [0.39355704, 0.09248306],
    [0.35133842, 0.54441284],
    [0.13681042, 0.17548928],
    [0.04615694, 0.97293731],
    [0.86735196, 0.36746758],
    [0.98960875, 0.15649625],
    [0.24814803, 0.2196556 ],
    [0.48686223, 0.49917513],
    [0.18851225, 0.34901664],
    [0.34089142, 0.6654274 ],
    [0.66748752, 0.88006977],
    [0.51197723, 0.54342045],
    [0.78116126, 0.92042426],
    [0.28543961, 0.82866196],
    [0.90937475, 0.80235109],
    [0.18206596, 0.53625329],
    [0.86331816, 0.5091636 ],
    [0.32643995, 0.76884931],
    [0.63066172, 0.13450645],
    [0.36346742, 0.17908587],
    [0.99641836, 0.53150963],
    [0.13792324, 0.65063333],
```

[0.04802794, 0.00694242],
[0.72190202, 0.32731276],
[0.45692652, 0.83247441],
[0.24375988, 0.93981671],
[0.87514053, 0.03333916],
[0.70456163, 0.81452708],
[0.16007477, 0.45625337],
[0.16840887, 0.27858991],
[0.04319724, 0.23593324],
[0.14555411, 0.4324552],
[0.78617344, 0.60345692],
[0.64402943, 0.07873906],
[0.01245088, 0.66012442],
[0.26540112, 0.34428836],
[0.22611285, 0.87237545],
[0.0419159, 0.19740167],
[0.96697376, 0.48545717],
[0.3046904, 0.7119917],
[0.20672657, 0.97945315],
[0.50284686, 0.54035019],
[0.11229302, 0.87331729],
[0.93920745, 0.57328742],
[0.49027584, 0.18948092],
[0.93154776, 0.94203961],
[0.80959092, 0.73613315],
[0.57975964, 0.97881427],
[0.16817448, 0.16737681],
[0.44390409, 0.06385818],
[0.02969398, 0.73999811],
[0.51516676, 0.27675307],
[0.27969742, 0.24194334],
[0.80786695, 0.6972817],
[0.84248551, 0.58704558],
[0.65241904, 0.05019886],
[0.52024481, 0.59387577],
[0.10131799, 0.19017301],
[0.8882563, 0.7663618],
[0.3400873, 0.05875461],
[0.70409154, 0.51999242],
[0.72484775, 0.97223298],
[0.32334477, 0.53447844],
[0.27358872, 0.62548276],
[0.0116032, 0.13010733],
[0.94105535, 0.24587023],
[0.46439725, 0.40732409],
[0.03028331, 0.78491114],
[0.96885651, 0.12682367],
[0.17888398, 0.09662701],
[0.58051206, 0.31594279],
[0.66428182, 0.55112451],
[0.84145128, 0.86356975],
[0.20242553, 0.98600577],

[0.77337493, 0.37536419],
[0.31619665, 0.0239244]])

we get as a figure



with the rectangular range defined as follows :

X = [0.2, 0.5] and
Y = [0.1, 0.8]

The set of points located inside this range is the following

[0.47544821 0.55444864]
[0.24814803 0.2196556]
[0.36346742 0.17908587]
[0.26540112 0.34428836]
[0.2475788 0.23090367]
[0.27969742 0.24194334]
[0.46439725 0.40732409]
[0.31688486 0.76546225]
[0.27358872 0.62548276]
[0.3046904 0.7119917]
[0.32334477 0.53447844]
[0.35133842 0.54441284]
[0.34089142 0.6654274]
[0.32643995 0.76884931]
[0.49027584 0.18948092]
[0.48686223 0.49917513]

THANK YOU!!!!!